



Audit of Lighter's Block Circuits

Date: August 4th, 2025

On August 4th, 2025, Lighter tasked zkSecurity with auditing its Plonky2 circuits. The audit lasted three weeks with two consultants. During the engagement, the team was granted access to the relevant parts of the Lighter codebase via a custom repository. Additionally, a [whitepaper](#) outlining various aspects of the protocol was also shared.

Scope

The scope of the audit included the following components:

- All top-level circuits in the Prover's `circuit/src/` directory.
- All circuits in `circuit/src/types` and `circuit/src/transactions`.
- The low-level circuit in `circuit/src/bigint/unsafe_big/mod.rs`.

Summary and recommendations

Overall, we found the codebase to be well-structured and tested. Unfortunately, both the documentation and the whitepaper were outdated and did not reflect the current state of the codebase.

Apart from the concrete findings listed at the end of this report, we have some strategic recommendations for keeping the codebase secure in the longer term:

- **Document the state machine and valid transitions:** The state machine of the rollup is quite complex, and there is a high number of possible transactions. It would be beneficial to write a clear specification of the valid state transitions, together with a description of the invariants that must be preserved during execution.
- **Minimize code duplication:** There are several instances of code duplication in the codebase, for example, different transactions that implement similar checks. Duplicated code is harder to maintain and can lead to inconsistencies long-term. We suggest refactoring the codebase to group similar checks into reusable gadgets.
- **Document the usage of the fields and the objects in the transaction state:** The account and order objects in the transaction state are used in different ways depending on the context. Additionally, these objects are used as both an input and an output of the transaction processing, which can be confusing and error-prone.
- **Be careful about empty objects that hash to zero:** Sparse Merkle trees are ubiquitous in the codebase. They are implemented with the common heuristic that an empty leaf hashes to zero. This allows for a standardized and efficient computation of the root for an empty tree of any depth for any data structure. However, if a leaf is considered empty only considering a subset of its fields, it can lead to non-determinism in reading a leaf from the current state, and important checks being skipped. We refer to our [finding below](#) for a concrete example of this issue.

Introduction

Lighter is an application-specific, EVM-compatible zk-rollup that implements a non-custodial exchange with a special focus on perpetual futures. Lighter's layer 2 (L2) state transitions are verified and finalized on a layer 1 (L1) chain — currently, Ethereum. The L1 ensures that a batch of L2 state transitions is valid by verifying a corresponding zero-knowledge proof (ZKP).

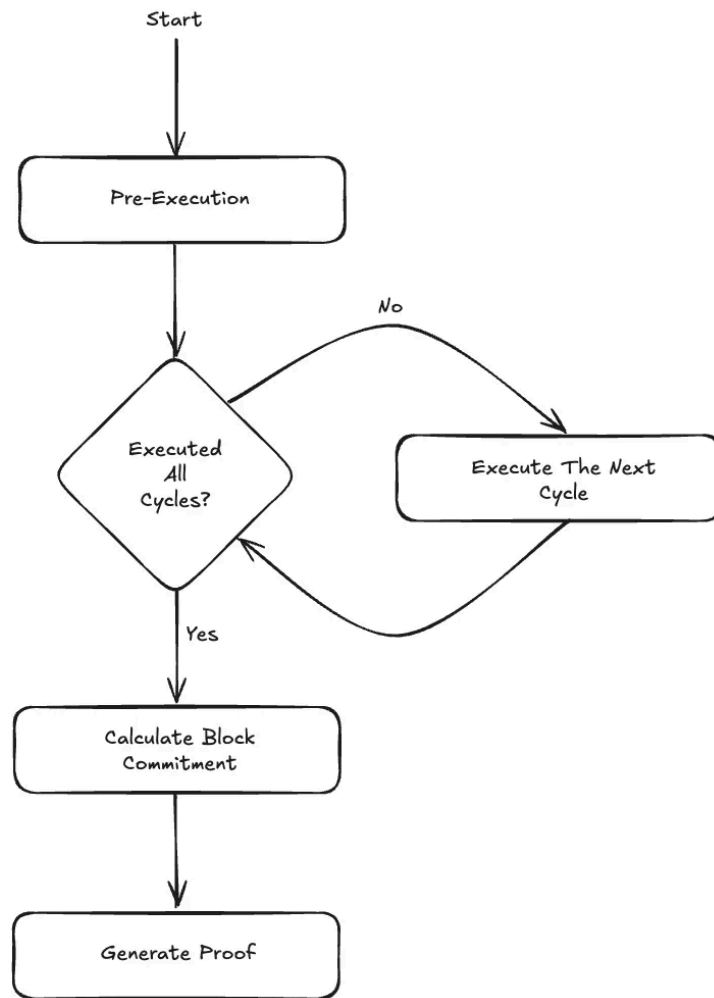
The role of the ZKP is twofold: to prove that the batch of L2 transitions is valid and to interact with the L1 contracts to expose relevant information about the transition batch. Some transitions require additional logic execution and policy enforcement on the L1. Additionally, the rollup implements a censorship-prevention mechanism, ensuring that the sequencer cannot censor the inclusion of transactions in the L2 state for too long; otherwise, the L1 contracts will stop normal operation and switch to a "desert mode," where users can withdraw their funds directly from the contracts.

There are three main components that make up the Lighter protocol: the Sequencer, the Prover, and the Smart Contracts. The **Sequencer** is the primary interface for users: it organizes transactions into structured blocks and commits these blocks to the L1 for immutable storage. The **Prover** takes the current state of the rollup and a batch of user transactions as inputs, processing them to generate a proof of execution. These proofs are used to verify the correctness of the new state, ensuring that it is the result of a correct execution of the specified transactions. Lastly, the **Smart Contracts** act as a settlement layer for the rollup, maintaining the integrity of the system by storing block and state root information and by verifying the block-execution proofs to correctly update the state.

A detailed introduction to Lighter's protocol has already been created during our previous audit engagement, we refer the reader to [this report](#). Since there have been some changes (most notably, the circuits have been migrated from Gnark to Plonky2), we also give an overview of the main components of the circuits below.

Overview of the circuits

The current audit was focused on Lighter's Prover component, which implements the verifiable matching and liquidation engines that generate block-execution proofs through its arithmetic circuits. For each block, the Prover generates a proof for the execution of a set of transactions. Each block encompasses a pre-execution phase and several execution cycles. Each transaction is either executed in a single execution cycle or unrolled into multiple execution cycles. Once all the cycles are executed, the arithmetic circuits compute a block commitment, which is the composite hash of the old and new state roots, as well as the block and transaction data. The Prover then generates a proof for the execution of the corresponding block, where the block commitment is the public input used for proof verification.



Special emphasis was placed on the following circuits:

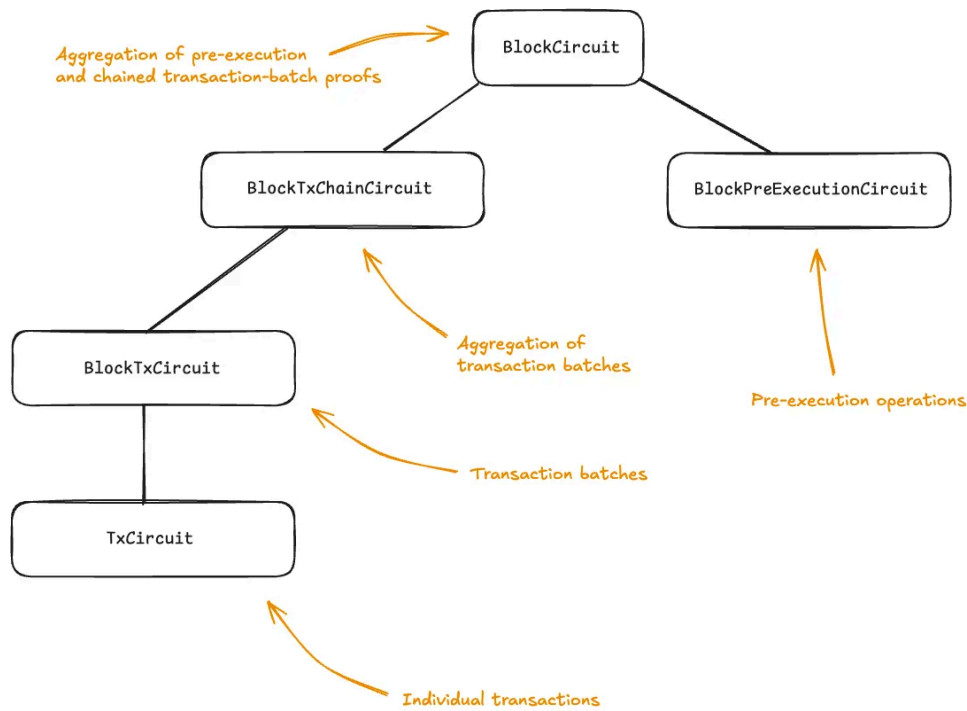
BlockPreExecutionCircuit. Handles block-level pre-execution operations like funding rate calculations and oracle-price processing.

BlockTxCircuit. Processes a batch of transactions within a block (at the time of writing, batches of six transactions).

BlockTxChainCircuit. Aggregation layer that implements recursive proof composition of transaction batches. In particular, it performs “IVC-style” aggregation, recursively verifying a proof for `BlockTxCircuit` for the $n + 1$ block, and verifying a proof for `BlockTxChainCircuit` for the n batch. The resulting proof is a `BlockTxChain` proof for the $n + 1$ block. Notice that this design allows the block to contain an arbitrary number of transactions.

BlockCircuit. Top-level circuit that recursively verifies a pre-execution and transaction-processing proofs into a block proof.

The “block-circuit hierarchy” can be depicted as follows:



Block Circuits

The **BlockPreExecutionCircuit** is responsible for processing block-level operations that occur before individual transaction execution. It maintains important economic parameters of Lighter's trading platform through three primary mechanisms:

1. Oracle price updates
2. Premium calculations
3. Funding rate calculations

Since perpetual futures do not have an expiry date, the price of a perpetual-future contract can drift away from the underlying asset's spot price if left unchecked. To fix this, Lighter implements a periodic funding mechanism: a fee that's periodically exchanged between longs and shorts to maintain price stability and avoid significant deviations of perpetual-future prices from the spot price. When the funding rate is positive, users with long positions pay a funding fee to users with short positions. When the funding rate is negative, users with short positions pay a fee to users with long positions. These payments are fully peer-to-peer with no fees taken by the exchange.

To determine the index price (which is equivalent to the spot price), Lighter currently uses a combination of oracles, with [Stork](#) as their main oracle provider.

One crucial quantity for the computation is the 1-hour premium, which is computed as follows: every minute, Lighter calculates the premium of each market, i.e., the differentiation of the mark price from the index price via:

$$\text{premium}_t = \frac{\text{Max}(0, \text{Impact Bid Price}_t - \text{index}_t) - \text{Max}(0, \text{index}_t - \text{Impact Ask Price}_t)}{\text{index}_t}$$

At the end of each hour, the 1-hour premium is then calculated as the time-weighted average of the 60 premiums calculated over the last hour.

$$\text{1h-premium} = \text{TWAP of premium}_t \text{ since the latest funding}$$

Additionally, each market has a fixed interest rate component that accounts for the difference in interest rates of the base and quote currencies. Together with the aforementioned 1-hour premium, the funding rate is then calculated via:

$$\text{funding rate} = \text{interest rate component} + \text{1h-premium}/8$$

Notice that division by 8 ensures that funding payments for the premium are distributed over 8 hours, aligning Lighter with the funding approach of most centralized perpetual exchanges. The funding round payment for account i and market j is then calculated as follows:

$$\text{funding}_{\text{account}_{i,j}} = (-1) \times \text{position}_{\text{account}_{i,j}} \times \text{mark}_j \times \text{funding rate}_j$$

While pre-execution handles block-level economic parameters, the `BlockTxCircuit` handles individual transactions using that economic context, i.e., it performs the actual trading operations.

The circuit is responsible for processing a configurable batch of (currently 6) transactions with each transaction receiving the current state and producing an updated state that becomes the input for the next transaction. The core part of this process is the following loop in the circuit's

`define_tx_loop` function:

```

// For each transaction, the circuit calls the transaction's "define" method,
// which processes the transaction and returns the updated state
for (index, tx) in self.target.txs.iter_mut().enumerate() {
    let (
        tx_priority_operations_pub_data,
        priority_operations_pub_data_exists,
        tx_on_chain_operations_pub_data,
        on_chain_pub_data_exists,
        register_stack_after,
        all_market_details_after,
        account_tree_root_after,
        account_metadata_tree_root_after,
        market_tree_root_after,
    ) = tx.define(
        index,
        chain_id,
        &mut self.builder,
        self.target.created_at,
        &current_register_stack,
        &current_all_market_details,
        current_account_tree_root,
        current_account_metadata_tree_root,
        current_market_tree_root,
    );

    current_register_stack = register_stack_after;
    current_all_market_details = all_market_details_after;
    current_account_tree_root = account_tree_root_after;
    current_account_metadata_tree_root = account_metadata_tree_root_after;
    current_market_tree_root = market_tree_root_after;

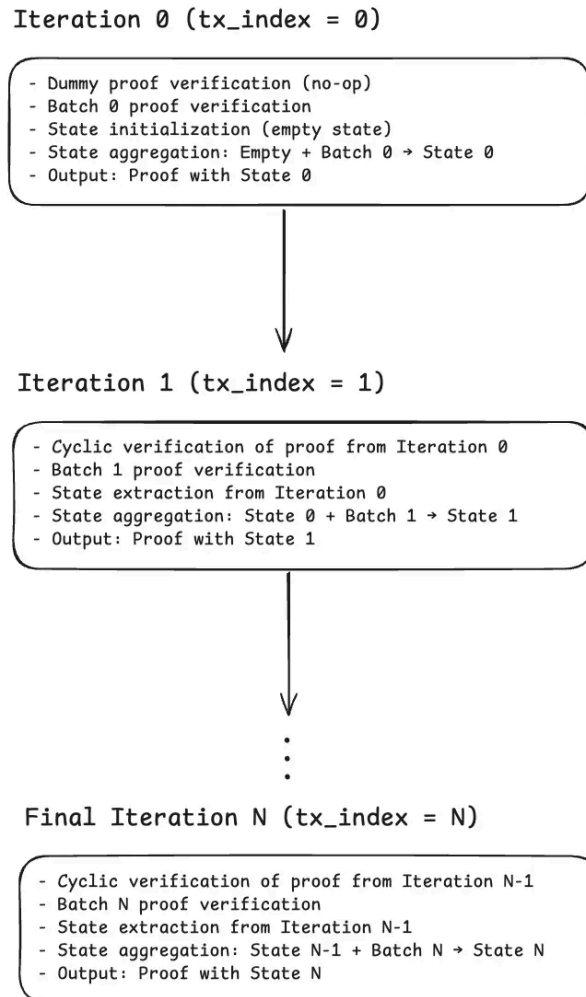
    on_chain_operations_count = self
        .builder
        .add(on_chain_operations_count, on_chain_pub_data_exists.target);
    on_chain_operations_pub_data = self.builder.select_arr_u8(
        on_chain_pub_data_exists,
        &tx_on_chain_operations_pub_data,
        &on_chain_operations_pub_data,
    );

    priority_operations_count = self.builder.add(
        priority_operations_count,
        priority_operations_pub_data_exists.target,
    );
    priority_operations_pub_data = self.builder.select_arr_u8(
        priority_operations_pub_data_exists,
        &tx_priority_operations_pub_data,
        &priority_operations_pub_data,
    );
}

```

The batch proofs generated by this circuit are subsequently aggregated through the `BlockTxChainCircuit`. This circuit is a recursive proof aggregation circuit that is responsible for combining the batch proofs from `BlockTxCircuit` into a single aggregated proof that can subsequently be combined with the proof from the pre-execution circuit. The cyclic proof composition allows the Lighter prover to process an arbitrary number of batches while maintaining a constant proof size for the whole block.

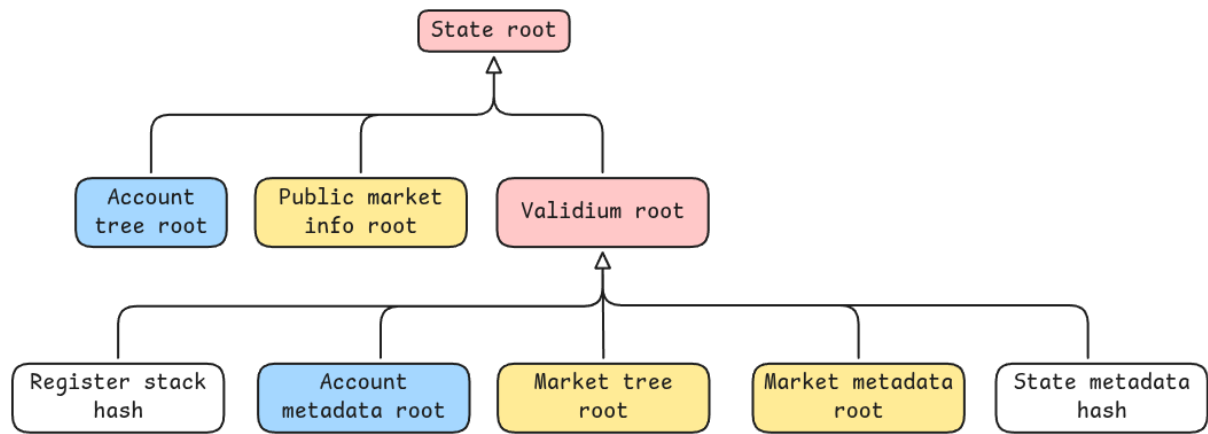
On a high level, the aggregation flow of the circuit can be thought of as follows:



The `BlockCircuit` serves as the final step in the block processing pipeline. After the `BlockPreExecutionCircuit` has handled the economic parameters (funding rates, premiums, oracle prices) and the `BlockTxChainCircuit` has aggregated all the batch proofs, the `BlockCircuit` takes the outputs of these two circuits and combines them into a single block proof.

Rollup State Tree

The state tree is a combination of multiple Merkle and sparse Merkle trees, which are used to store all the information about the rollup state. The state tree hash is a cryptographic hash that uniquely represents the entire state of the rollup at a given point in time.



We briefly describe the main components of the state tree.

Account information is stored in two separate sparse Merkle trees, rooted respectively in the account tree root and the account metadata tree root.

- For each account, relevant information is stored in the account tree, such as the collateral, the positions, and the public pool shares owned by the account.
- The account metadata tree stores additional information about the account, such as the number of orders, the DMS switch time, the API keys tree root, and the account order tree root.

Worthy of note is the account order tree, which is a Merkle tree that stores all the orders for a given account. Orders are stored in two positions, named `index0` and `index1`, which are computed using two different strategies.

- `index0` is computed using the first 8 bits as the market index and the last `ORDER_NONCE_BITS` bits as the order nonce.
- `index1` is computed using a hash of a client order ID of size `CLIENT_ORDER_ID_BITS`, which is provided by the user when creating the order. Notice that `ORDER_NONCE_BITS` is equal to `CLIENT_ORDER_ID_BITS`, so both indices are of the same size.

Both orders live in the same tree but in separate parts. The account order index is computed first by choosing the 8-bit market index and then adding the order nonce.

```
account_order_index = (market_index + 1) * (1 << ORDER_NONCE_BITS) + order_nonce
```

The market index is incremented by one because market index 0 is reserved for client order IDs, ensuring that the two index spaces are disjoint.

For a description of the order book tree, we refer to the previous audit report, since the data structure is unchanged.

Transaction Processing

Transactions are the means by which the rollup state is modified. The engine that checks the validity of transactions takes as input the current state tree of the rollup and a transaction and produces as output a new state tree. Reading from the state is done by verifying Merkle proofs for the relevant objects involved in the current transaction, and writing to the state is done by updating the objects and recomputing the Merkle roots up to the state root.

Overview of Transaction Types

Transactions are divided into three main categories:

- **Internal transactions:** These are transactions that are enqueued by the sequencer automatically and are either “system” transactions used to execute the unrolled instructions or are sent by the sequencer when certain triggers are hit (e.g., a liquidation or a stop-loss order).
- **L1 transactions:** These are transactions that are sent by users on L1 and are enqueued in the priority queue. They are executed in the order they are enqueued, and a block can contain at most one L1 transaction.
- **L2 transactions:** These are transactions that are sent by users to the sequencer and are enqueued by the sequencer. A block can contain an arbitrary number of L2 transactions.

We now give an overview of all the transaction types.

Internal transactions

Order internal transactions:

- **Internal cancel all orders:** cancels all orders for a given account. This can be enqueued either because the account is in liquidation, or because the dead man switch should be triggered. This transaction pushes a `CANCEL_ALL` instruction on the register stack.
- **Internal cancel order:** cancels a single order for a given account. This is the result of either a single cancel order instruction, or a “cancel all” instruction. If it is part of an unrolled “cancel all”, the register stack is modified to contain the remaining orders to cancel. If there are no more orders to cancel, the register stack is popped.
- **Internal claim order:** this transaction is used to continue the matching of a taker order that was partially matched in the previous cycle. The register stack is modified by the matching engine to contain the remaining size of the taker order to match, or it is popped if the order was fully matched in this cycle.
- **Internal create order:** this transaction is used to insert an order that was previously submitted. This can be either the result of a triggered order by price (e.g., stop-loss or take-profit), a triggered order by time (in TWAP orders), or a trigger of a child order (in OTO/OCO/OTOCO orders).

Risk management internal transactions:

- **Internal liquidate position:** this transaction is used to liquidate an account whose health is in “partial liquidation”. The transaction will try to improve the account’s health by inserting an immediate-or-cancel order at the “zero price”, which is the special price for which the ratio

between the maintenance margin requirement and the account value stays the same after the trade.

- **Internal deleverage:** this transaction is used to perform a deleverage of an account that is in bankruptcy. The sequencer will select a counterparty account that has a position in the opposite direction, and will forcefully apply a trade between the two accounts. The deleverager account can be either an insurance fund, or a regular user selected from the automatic deleverage queue.
- **Internal exit position:** this transaction is enqueued when a market has expired, and it forces the account to exit its position at the latest price.

L1 transactions

Deposit and withdraw transactions:

- **L1 Deposit:** this transaction is used to deposit funds into a (possibly new) account. The accepted funds are added to the account's collateral. The sequencer may accept fewer funds than are sent.
- **L1 Withdraw:** this transaction is used to withdraw funds from the L2 by an account. The requested funds are subtracted from the account's collateral and posted on-chain as a "withdraw" operation.

Account and order management transactions:

- **L1 Create Order:** this transaction is used to create a new order for a given account. The transaction pushes an `INSERT_ORDER` instruction on the register stack.
- **L1 Cancel All Orders:** this transaction is used to cancel all orders for a given account. This transaction pushes a `CANCEL_ALL` instruction on the register stack.
- **L1 Change Pubkey:** this transaction is used to change one of the public keys (also known as API keys) of an account or to create a new one.
- **L1 Burn Shares:** this transaction is used to burn the pool shares of an account. The account receives its portion of the total pool value in exchange for burning the shares.

Market management transactions:

- **L1 Create Market:** this transaction is used to create a new market. This transaction can only be executed by the protocol's governor, which is enforced by the L1 contracts.
- **L1 Update Market:** this transaction is used to update the parameters of a market. This transaction can also only be executed by the protocol's governor.

L2 transactions

Funds-transferring transactions:

- **L2 Transfer:** this transaction is used to transfer funds between two accounts.
- **L2 Withdraw:** L2 version of the withdraw transaction.

Order transactions:

- **L2 Create Order:** L2 version of the create order transaction.
- **L2 Create Grouped Orders:** this transaction is used to create a group of orders that are linked together. The order group type can be either OCO, OTO, or OTOCO.
- **L2 Modify Order:** this transaction is used to modify an existing order.
- **L2 Cancel Order:** this transaction is used to cancel a single order for a given account.
- **L2 Cancel All Orders:** L2 version of the cancel all orders transaction.

Public pool transactions:

- **L2 Create Public Pool:** this transaction is used to create a new public pool account, which can be either an insurance fund or a public pool. The account that creates the pool becomes the operator of the pool, deposits some funds in the pool, and receives the initial operator shares.
- **L2 Update Public Pool:** this transaction is used to update the parameters of a public pool. Only the operator of the pool can execute this transaction.
- **L2 Mint Shares:** this transaction is used to mint new shares for a public pool. The account that executes the transaction deposits some funds in the pool and receives the corresponding shares.
- **L2 Burn Shares:** L2 version of the burn shares transaction.

Margin and leverage transactions:

- **L2 Update Leverage:** this transaction is used to update the leverage parameters of an account for a market: the account can change between cross-margin and isolated margin, and can also change the initial margin fraction.
- **L2 Update Margin:** this transaction is used to update the individual allocated margin for the account positions in the case of isolated margin. The user can either increase or decrease the allocated margin for a given market.

Account management transactions:

- **L2 Change Pubkey:** L2 version of the change pubkey transaction.
- **L2 Create Sub-Account:** this transaction is used to create a new sub-account for an existing account.

Transaction Loop and the Register Stack

The instruction register stack is a data structure that stores the current state of execution of the current transaction. At the beginning of each block, in the pre-execution step, the register stack is ensured to contain an `EXECUTE_TRANSACTION` instruction in its head. The `EXECUTE_TRANSACTION` instruction is a special instruction type that signals to the rollup engine that all previous pending execution instructions have been completed and that a new transaction can be executed.

```
// Register stack should be in Execute Mode when pre-execution is in progress
let execute_transaction = self.builder.constant_from_u8(EXECUTE_TRANSACTION);
let is_register_instruction_type_execute = self.builder.is_equal(
    self.target.register_stack_before[0].instruction_type,
    execute_transaction,
);
// ...
self.builder.conditional_assert_true(is_pre_block_execution,
is_register_instruction_type_execute);
```

The instruction register stack is an array of `BaseRegisterInfoTarget` objects. Instructions that are pushed on the stack need to be executed before processing the next transaction, while instructions that are popped from the stack are considered completed. The engine enforces that, if the stack is not empty, only the correct transaction type can be executed. For example, if the top instruction on the stack is an `INSERT_ORDER` instruction, that means that a partial matching is in progress, and the next enqueued transaction must be an internal claim order transaction to continue the matching of the order.

Each `BaseRegisterInfoTarget` object is represented in the following structure.

```
pub struct BaseRegisterInfoTarget {
    pub instruction_type: Target,
    pub market_index: Target,
    pub account_index: Target,

    pub pending_size: Target,

    pub pending_order_index: Target,
    pub pending_client_order_index: Target,
    pub pending_initial_size: Target,
    pub pending_price: Target,
    pub pending_nonce: Target,
    pub pending_is_ask: BoolTarget,

    pub pending_type: Target,
    pub pending_time_in_force: Target,
    pub pending_reduce_only: Target,
    pub pending_expiry: Target,

    pub generic_field_0: Target,

    pub pending_trigger_price: Target,
    pub pending_trigger_status: Target,
    pub pending_to_trigger_order_index0: Target,
    pub pending_to_trigger_order_index1: Target,
    pub pending_to_cancel_order_index0: Target,
}
```

We briefly describe some of the most interesting fields of the instruction register.

- The `instruction_type` field is used to determine the type of instruction being processed. In particular:
 - `EXECUTE_TRANSACTION` is used to indicate that no more instructions are to be processed and the next transaction can be whatever (L1/L2/internal).
 - `INSERT_ORDER` is used to insert a new order into the order book.
 - `CANCEL_ALL_ACCOUNT_ORDERS` , `CANCEL_SINGLE_ACCOUNT_ORDER` , `CANCEL_POSITION_TIED_ACCOUNT_ORDERS` , `CANCEL_ALL_CROSS_MARGIN_ORDERS` , and `CANCEL_ALL_ISOLATED_MARGIN_ORDERS` are used to cancel orders.
 - `TRIGGER_CHILD_ORDER` is used to trigger a child order in an OTO order.
- The `pending_size` field represents a generic execution amount left to process during instruction unrolling, with different meanings per instruction type.
 - For `INSERT_ORDER` instructions, it represents the remaining order size/quantity in base units.
 - For `CANCEL_ALL` instructions, it represents the count of orders still to cancel.
- The `generic_field_0` field is a generic field that is used to store the slippage accumulator for market orders during matching.

Matching Engine

The matching engine receives as input multiple objects, which are stored in the `tx_state` object.

The **head of the register stack** contains information about the taker order to be matched or inserted in the book. Notice that each match cycle only matches a taker order with at most one maker order, so if the match is partial, the remaining size of the taker order is left in the register stack as an `INSERT_ORDER` instruction, and the sequencer will subsequently sequence an "internal claim order" transaction to continue matching the taker order. There are then the **order** and the **account order** objects, which are used for different purposes in the engine.

Initial matching engine setup.

Throughout the implementation, there are four main boolean flags that are used to control the effect of the matching engine:

- `update_status_flags` : this flag acts as a general switch to enable or disable subsequent updates to the matching engine logic. It can be thought of as some sort of "unrolled early return"; that is, once it is set to false, all subsequent logic of the matching engine is disabled. It is worth noting that this flag is initially set to `tx_state.matching_engine_flag` , which is a flag that is set to `true` by transactions that require the activation of the matching engine. By default, it is set to `false` for all transactions.
- `cancel_taker_order` : this flag is set to `true` if the taker order should be cancelled after the matching process. This is done by just executing a pop from the instruction stack.
- `cancel_maker_order` : this flag is set to `true` if the maker order should be cancelled after the matching process, for example, because it has been fully matched by the taker order or

because it has expired.

- `insert_taker_order` : this flag is set to `true` if the taker order should be inserted into the state after the matching process.

First, if the pending order trigger status is not `NA`, that means that the order has to be inserted into the pending order tree, and subsequent matching logic is disabled.

If the order is a "reduce only" order, then the direction of the order must be opposite to the position direction. That is, for an ask order, the overall position must be long, and for a bid order, the overall position must be short. If this is not the case, then the order is cancelled.

The `order` field in the `tx_state` contains an order index that depends on the context:

- if there is a matching order, then it is the maker order index
- if there is no matching order, then it is enforced to be the taker order index

In either case, the order index is used to compute if this is the best match in the order book, using a price-priority mechanism, as described in the next section.

Ensuring the best match in the order book.

The matching engine uses the `get_quote` function to get the size and quote of the orders with higher priority than the one currently being matched. The output of this function is enforced to be zero, which enforces that the matched order is the best order in the book.

The mechanism is as follows: the function walks through the path for the current order, starting from the leaf order. For each internal node, it computes the difference between the current size sum and the child node. This quantity corresponds to the size of the orders in the "sibling subtree" with respect to the child node. Now, there are two scenarios, depending on whether the current child in the path is a left or right child:

1. If the sibling subtree contains orders with higher priority than the current order, then the size of the orders in the sibling subtree is added to the accumulator.
2. Otherwise, it is ignored, as the orders in the sibling subtree have a lower priority than the current order.

At the end, the accumulator contains the size of the orders with higher priority than the current order, and it is enforced to be zero. We refer to the previous audit report for more details on how to compute the size and quote of the orders with higher priority.

We note that this computation works with either a matching order index or the taker order index: in either case, the returned quantity will be the amount of the orders that could have matched the current order, giving a better price priority than the current match.

Handling of self-trade orders.

Self-trade orders are orders that match with an order from the same account. In this case, most of the logic is skipped, since a self-trade does not change positions. The only way a self-trade can fail is if the maker order has expired or if the pending order is a post-only order.

Slippage logic for market orders

On market orders, the user can specify an execution price, which is stored in the `pending_price` field of the register. The matching engine has to guarantee that the average execution price of the order is not worse than the specified price.

To do so, the engine keeps track of the current slippage of the order in the `generic_field_0` field of the register. This field is initially set to zero and keeps track of the current slippage amount, which is the price difference in favor of the market order, weighted by the size of the executed orders so far. To compute how much size can be safely executed without incurring too much slippage, the engine computes the integer division `generic_field_0 / price_diff`, where `price_diff` is the difference between the matched price and the price of the maker order, signed according to the direction of the order. At the end of the matching process, if there has been a slippage in the matching, this field is updated accordingly.

To explain the mechanism, we consider the following example, which was provided by the Lighter team. Suppose in the book there are two orders:

- Ask order of size 10 at price 100
- Ask order of size 10 at price 120

The user submits a bid market order of size 20 at price 105.

The matching engine will first match the first ask order of size 10 at price 100. In this case, no slippage is incurred because the price of the market order is better than the price of the maker order. The `generic_field_0` is updated to $5 \cdot 10 = 50$.

Then, the second ask order of size 10 at price 120 is matched. In this case, there is slippage because the price of the market order is worse than the price of the maker order. However, since we matched 10 units at price 100 before, we can still execute a partial match of the market order without making the average execution price worse than the specified price. In this second match, the price difference is `15`, and the size of the executed order is computed as $\lfloor 50/15 \rfloor = 3$.

The average execution price is computed as

$$\text{average execution price} = \frac{100 \cdot 10 + 120 \cdot 3}{10 + 3} = \frac{1000 + 360}{13} = \frac{1360}{13} \approx 104.6$$

which is better than the specified price of 105 for a bid market order, so the overall execution is valid.

Order Groups

Order groups are a set of orders that are linked together: they can be either OCO, OTO, or OTOCO.

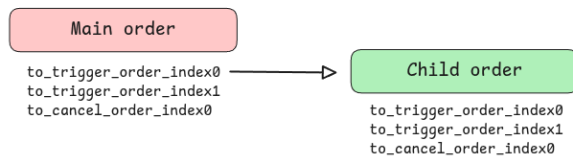
- **OTO (One triggers the other):** whenever the main order is filled, the child order is triggered and inserted. The main order can be either a limit or market order, while the child order must be a conditional order (e.g., stop-loss or take-profit).
- **OCO (One cancels the other):** whenever one of the two orders is triggered and inserted, the other is cancelled. In this case, both orders have to be conditional orders, and they must be

opposite in direction (i.e., one is a stop-loss and the other is a take-profit).

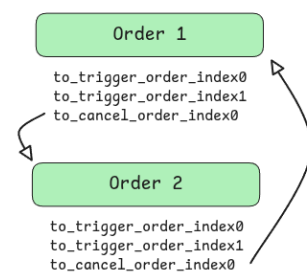
- **OTOCO (One triggers the other, one cancels the other):** this is a combination of OTO and OCO orders. Whenever the main order is filled, two other orders are triggered and inserted, and they are linked together in an OCO group. This is used to simultaneously set a stop-loss and a take-profit order after entering a position.

In the circuits, order groups are handled using three index pointers in the order's structure. The `pending_to_trigger_order_index0` and `pending_to_trigger_order_index1` fields are used to store the child orders that will be triggered when the current order is filled. The `pending_to_cancel_order_index0` field is used to store the order that will be cancelled by the current order. The pointer structure enforced by the "L2 Create Grouped Orders" transaction is depicted below.

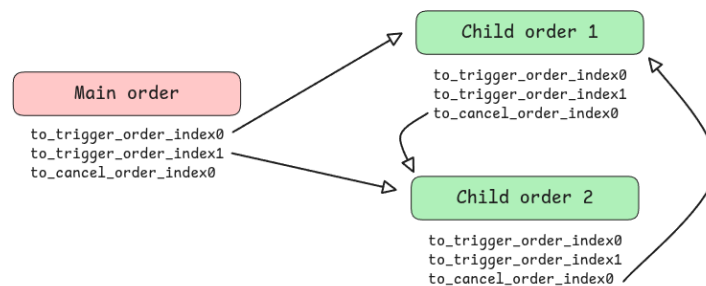
OTO (One triggers the other)



OCO (One cancels the other)



OTOCO (One triggers a one cancels the other)



Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than high-severity findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

ID	COMPONENT	NAME	RISK
#00	tx_constraints.rs	New accounts can be initialized with arbitrary collateral and positions due to missing checks	High
#01	block_tx_constraints.rs	On-chain operation count can bypass the block limit, leading to denial of L1 withdrawals	High
#02	uint/u16/split.rs	Splitting a BigInt into chunks of 16 bits is affected by aliasing	Medium
#03	transactions/l1_change_pubkey.rs	The L1 Change Public Key transaction does not check that the API key index is not nil	Low
#04	block_pre_execution_constraints.rs, matching_engine.rs	Inconsistent is_ask semantics lead to swapped impact prices and wrong premium formula	Low
#05	transactions/l2_cancel_all_orders.rs	Scheduled cancel-all time lower bound is not enforced and comparison assumptions are not satisfied	Low

ID	COMPONENT	NAME	RISK
#06	circuit/src/	Inconsistent usage of constants, commented-out code, and lingering TODOs	Informational
#07	types/Register/register_stack.rs	Executing a pop operation on an empty stack could lead to an invalid state	Informational
#08	transactions/l2_change_pubkey.rs	New API keys are not checked to be initialized with a zero-nonce	Informational
#09	tx_constraints.rs	The head of the register stack is addressed using the wrong constant	Informational

#00 - New accounts can be initialized with arbitrary collateral and positions due to missing checks

Severity: High Location: tx_constraints.rs

Description. The L1 deposit transaction allows for the creation of new accounts by depositing some amount of collateral. The transaction will store the corresponding leaf in the account tree. An account is considered "new" if its field `master_account_index` is zero.

```
pub fn is_new_account(&self, builder: &mut Builder) → BoolTarget {
    let is_special_account = self.is_special_account(builder);
    let is_master_account_empty = builder.is_zero(self.master_account_index);
    builder.and_not(is_master_account_empty, is_special_account)
}
```

Hashing the account structure returns zero if the account is new, regardless of the values of all the other fields.

```
fn hash_from_partial_hash(
    &self,
    builder: &mut Builder,
    partial_hash: &HashOutTarget,
) → HashOutTarget {

    // ...

    let empty_hash = builder.zero_hash_out();
    let is_new_account = self.is_new_account(builder);

    builder.select_hash(is_new_account, &empty_hash, &non_empty_hash)
}
```

This means that, when reading a new account from the state tree, all other fields are not included in the hash, and the prover can set an arbitrary value which will pass the Merkle proof check. During an account creation, only three fields are set in the newly created account: `l1_address`, `master_account_index`, and `account_type`.

```

tx_state.accounts[OWNER_ACCOUNT_ID].l1_address = builder.select_biguint(
    should_update_for_new_account,
    &self.l1_address,
    &tx_state.accounts[OWNER_ACCOUNT_ID].l1_address,
);

tx_state.accounts[OWNER_ACCOUNT_ID].master_account_index = builder.select(
    should_update_for_new_account,
    self.account_index,
    tx_state.accounts[OWNER_ACCOUNT_ID].master_account_index,
);

tx_state.accounts[OWNER_ACCOUNT_ID].account_type = builder.select(
    should_update_for_new_account,
    master_account_type,
    tx_state.accounts[OWNER_ACCOUNT_ID].account_type,
);

```

Most importantly, the collateral is updated by adding the deposited amount to the existing collateral, without checking if the account is new.

```

let collateral_after = builder.add_bigint_non_carry(
    &tx_state.accounts[OWNER_ACCOUNT_ID].collateral,
    &BigintTarget {
        abs: collateral_delta,
        sign: collateral_delta_sign,
    },
    BIG_U128_LIMBS,
);

```

Additionally, the `positions` field is never set during account creation, and can be initialized to an arbitrary value by the prover.

Impact. A malicious prover can exploit this issue to create a new account with arbitrary collateral and positions, which could lead to a malicious L2 operator draining the protocol.

Recommendation. Implement checks to ensure that new accounts are initialized with valid collateral and positions. This would involve setting default values for these fields or requiring explicit initialization if the input account leaf is a new account. Additionally, we recommend being careful when hashing empty leaves, as we discussed in [our strategic recommendations section](#).

Client response. Acknowledged and fixed.

#01 - On-chain operation count can bypass the block limit, leading to denial of L1 withdrawals

Severity: High Location: block_tx_constraints.rs

Description. In the `BlockTxCircuit`, the definition of the transaction loop increments the priority operation and on-chain operation counters for each transaction that exposes the respective operation. After defining the transaction, the code checks whether the transaction is a priority operation and increments the counters accordingly. Additionally, if such data exists, the exposed public data gets overwritten with the current public data.

```
on_chain_operations_count = self
    .builder
    .add(on_chain_operations_count, on_chain_pub_data_exists.target);
on_chain_operations_pub_data = self.builder.select_arr_u8(
    on_chain_pub_data_exists,
    &tx_on_chain_operations_pub_data,
    &on_chain_operations_pub_data,
);

priority_operations_count = self.builder.add(
    priority_operations_count,
    priority_operations_pub_data_exists.target,
);
priority_operations_pub_data = self.builder.select_arr_u8(
    priority_operations_pub_data_exists,
    &tx_priority_operations_pub_data,
    &priority_operations_pub_data,
);
```

However, the `BlockTxChain` circuit **assumes that for each block, there is at most one priority operation and one on-chain operation**. This is not enforced by the circuits. In the `BlockTxChain` circuit, the exposed public data for on-chain operations is stored in a fixed-size array. To check that the count does not surpass the block limit, the code checks that the current count is not equal to the block limit, because it assumes that only one priority operation and one on-chain operation can be exposed per block.

```
// If current tx has an on-chain operation, then we should not have
on_chain_operations_limit operations before
// Assuming one on-chain operation on tx segment
let on_chain_operations_limit =
circuit.builder.constant_usize(on_chain_operations_limit);
circuit.builder.conditional_assert_not_eq(
    on_chain_operation_exists,
    block.on_chain_operations_count,
    on_chain_operations_limit,
);
```

Later, the new count is computed as the sum of the previous count and the current transaction's count.

```
on_chain_operations_count: circuit.builder.add(
    block.on_chain_operations_count,
    current_tx.on_chain_operations_count,
)
```

Impact. A malicious sequencer could surpass the limit of on-chain operations per block by enqueueing multiple transactions that expose on-chain operations in the same batch. This can lead to the prover not functioning properly, which can result in completeness issues of the rollup.

We note that, the public data for a whole block has fixed size, set to the `on_chain_operations_limit` and `priority_operations_limit`, respectively. The way the selection works is that, if the count is greater than the limit, then the current on-chain public data is ignored.

```
fn select_on_chain_pub_data(
    builder: &mut Builder,
    on_chain_operations_limit: usize,
    on_chain_operations_count: Target,
    on_chain_operations_pub_data: &mut [[U8Target;
ON_CHAIN_OPERATIONS_PUB_DATA_BYTES_SIZE]],
    tx_on_chain_operations_pub_data: &[U8Target;
ON_CHAIN_OPERATIONS_PUB_DATA_BYTES_SIZE],
    on_chain_operation_exists: BoolTarget,
) {
    assert_eq!(
        on_chain_operations_pub_data.len(),
        on_chain_operations_limit,
        "incorrect on chain pub data length"
    );

    (0..on_chain_operations_limit).for_each(|slot_id| {
        let slot_id_t = builder.constant_usize(slot_id);
        let is_current_slot = builder.is_equal(slot_id_t,
on_chain_operations_count);
        let flag = builder.and(on_chain_operation_exists, is_current_slot);

        let slot = on_chain_operations_pub_data.get_mut(slot_id).unwrap();
        for i in 0..ON_CHAIN_OPERATIONS_PUB_DATA_BYTES_SIZE {
            slot[i] = builder.select_u8(flag,
tx_on_chain_operations_pub_data[i], slot[i]);
        }
    });
}
```

We now suggest another attack that allows a malicious sequencer to keep operating normally, but enables it to deny L1 withdrawal requests. An L1 withdrawal transaction exposes both an on-chain operation and a priority operation public data:

- The on-chain operation is needed to update the L1 contract with the new balance of the L2 account.
- The priority operation is needed to satisfy the priority queue request, since this is an L1 transaction.

The attack works as follows:

1. The sequencer enqueues multiple transactions that expose on-chain operations, making `on_chain_operations_count` greater than the `on_chain_operations_limit`.
2. The sequencer now enqueues the L1 withdrawal transaction, which exposes both an on-chain operation and the priority operation.
3. On the L1 contracts, the priority queue request is satisfied, because the priority operation public data is correctly exposed. However, the corresponding on-chain operation public data is not exposed, leading to the L1 contract not being updated with the new balance of the L2 account.

Most importantly, after this attack, the L1 withdrawal transaction is correctly executed, leading to a correct change in the L2 state. However, the L1 contracts are not updated with the new balance, and the user cannot withdraw their funds. As a final note, this attack can be extended to any transaction that exposes an on-chain public data, for example, L2 withdrawals.

Recommendation. We recommend adding a boolean check to ensure that the priority operation and on-chain operation counters at the `BlockTx` level are not incremented if the respective operation has already been exposed in the current block, which can be done by adding a boolean check on the counters.

We shall note that this fix makes the rollup lose some completeness: the sequencer is forced to ensure that only one priority operation and one on-chain operation is enqueued per block, which could make some valid transaction sequences not provable.

Client response. Acknowledged and fixed.

#02 - Splitting a BigInt into chunks of 16 bits is affected by aliasing

Severity: Medium Location: uint/u16/split.rs

Description. The function `split_u64_to_u16s_le` is used to split a 64-bit field element into chunks of 16 bits each, represented as `U16Target` objects.

```
pub fn split_u64_to_u16s_le(&mut self, x: Target, num_limbs: usize) →
Vec<U16Target> {
    if let Some(cached) = self.u16_split_cache.get(&x) {
        return cached.to_vec();
    }

    let limbs = (0..num_limbs)
        .map(|_| self.add_virtual_u16_target_safe())
        .collect::<Vec<_>>();

    self.add_simple_generator(SplitToU16Generator {
        x,
        limbs: limbs.clone(),
        _phantom: PhantomData,
    });

    let mut acc = limbs.last().unwrap().0;
    let multiplier = self.constant_u64(1 << 16);
    for limb in limbs.iter().rev().skip(1) {
        acc = self.mul_add(acc, multiplier, limb.0);
    }
    self.connect(acc, x);

    self.u16_split_cache.insert(x, limbs.clone());

    limbs
}
```

The function witnesses the limbs, and then checks that:

$$x = \sum_{i=0}^3 \text{limbs}_i \cdot 2^{16i}$$

This check is done over the field, whose prime is the Goldilocks prime:

$$p = 2^{64} - 2^{32} + 1$$

If `num_limbs` is 4, the decomposition is not unique for values of `x` that are smaller than 2^{32} , because both the representation of x and the representation of $x + p$ are valid decompositions. This function is used in the unsafe bigint module to normalize an unsafe bigint into a `BigIntTarget`.

Impact. A malicious prover could exploit this aliasing issue to return an invalid normalized bigint that is different from the originally intended value.

Recommendation. We recommend implementing checks to ensure that the decomposition is unique, like it is done in the `split_u64_to_u32s_1e` function. It should suffice to check that, if the most significant two limbs are both equal to `0xffff`, then the two least significant limbs are both equal to `0`, similarly to what is done in the 32-bit case.

Client response. Acknowledged and fixed.

#03 - The L1 Change Public Key transaction does not check that the API key index is not nil

Severity: Low **Location:** transactions/l1_change_pubkey.rs

Description. In the L2 Change Public Key transaction, the public key index is enforced to be different from `NIL_API_KEY_INDEX` (which is currently equal to `0xff`).

```
// Can not fill NIL_API_KEY_INDEX  
let nil_api_key_index = builder.constant_from_u8(NIL_API_KEY_INDEX);  
builder.conditional_assert_not_eq(is_enabled, self.api_key_index,  
nil_api_key_index);
```

However, the analogous check is missing in the L1 Change Public Key transaction.

Impact. An L1 Change Public Key transaction with a nil API key index could be accepted, which could lead to unexpected behavior in the system.

Recommendation. Add a similar check in the L1 Change Public Key transaction to ensure the API key index is not nil.

Client response. Acknowledged and fixed.

#04 - Inconsistent `is_ask` semantics lead to swapped impact prices and wrong premium formula

Severity: Low **Location:** `block_pre_execution_constraints.rs`, `matching_engine.rs`

Description. The premium calculation implemented in `block_pre_execution_constraints.rs` differs from the intended formula specified in [Lighter's whitepaper](#). The expected formula is:

$$\text{premium}_t = \frac{\text{Max}(0, \text{Impact Bid Price}_t - \text{index}_t) - \text{Max}(0, \text{index}_t - \text{Impact Ask Price}_t)}{\text{index}_t}$$

But the circuit computes:

$$\text{premium}_t = \frac{\text{Max}(0, \text{Impact Ask Price}_t - \text{index}_t) - \text{Max}(0, \text{index}_t - \text{Impact Bid Price}_t)}{\text{index}_t}$$

At first glance, it appears that the premium is calculated incorrectly. However, further investigation shows that the root cause behind the different formulas lies in the semantics of the `is_ask` flag used in `get_quote` and `get_impact_price`. Passing `is_ask = true` to `get_impact_price` will compute the *bid* side, while `is_ask = false` will compute the *ask* side. See, for example, the following excerpt of `get_impact_price`:

```
let leaf_quote_amount = builder.select(is_ask, order.bid_quote_sum,
order.ask_quote_sum);
...
let total_quote_amount = builder.select(
    is_ask,
    order_path[ORDER_BOOK_MERKLE_LEVELS - 1].bid_quote_sum,
    order_path[ORDER_BOOK_MERKLE_LEVELS - 1].ask_quote_sum,
);
```

This inversion propagates into the computation of `impact_bid_price` and `impact_ask_price`, effectively swapping their values. In other words, `impact_bid_price` actually holds what should be the impact ask price, while `impact_ask_price` actually holds what should be the impact bid price. The flipped premium formula in the block pre-execution circuit compensates this and produces numerically correct premiums, albeit in a confusing and hard-to-maintain manner.

Impact. Premiums currently compute as expected only because two erroneous inversions cancel out. Any partial refactor (e.g., fixing the premium formula but not how `get_impact_price` handles `is_ask`) will break premium correctness.

Furthermore, the current implementation is misleading for developers. Future maintainers of the codebase could reasonably assume `is_ask = true` means “ask side,” or that `impact_bid_price` indeed stores a bid price, leading to incorrect code changes.

In particular, any downstream logic that reads `market_details.impact_bid_price` or `market_details.impact_ask_price` may misinterpret the values, causing unexpected logic errors.

Recommendation. Fix the logic of `get_impact_price` so that `is_ask = true` corresponds to the ask side and `is_ask = false` to the bid side. Moreover, update the premium calculation in the block pre-execution circuit to correctly reflect the formula that is given in the whitepaper.

Client response. Acknowledged and fixed.

#05 - Scheduled cancel-all time lower bound is not enforced and comparison assumptions are not satisfied

Severity: Low Location: transactions/l2_cancel_all_orders.rs

Description. In the L2 Cancell All Orders transaction, there is an attempt to verify that the current time is within the interval defined by `block_timestamp + [MIN_ORDER_CANCEL_ALL_PERIOD, MAX_ORDER_CANCEL_ALL_PERIOD]`.

```
// Verify that while setting schedule cancel all, time is in range
block_timestamp + [MIN_ORDER_CANCEL_ALL_PERIOD, MAX_ORDER_CANCEL_ALL_PERIOD],
// otherwise zero
let max_order_cancel_all_period =
builder.constant_i64(MAX_ORDER_CANCEL_ALL_PERIOD);
let validity_time_end = builder.add(tx_state.block_timestamp,
max_order_cancel_all_period);
let is_enabled_and_scheduled_cancel_all = builder.and(is_enabled,
is_scheduled_cancel_all);

builder.register_range_check(self.time, TIMESTAMP_BITS);
builder.conditional_assert_lte(
    is_enabled_and_scheduled_cancel_all,
    self.time,
    validity_time_end,
    TIMESTAMP_BITS,
);
```

There are two issues with this code:

- First, the lower bound is not checked at all, so the time could be less than `block_timestamp + MIN_ORDER_CANCEL_ALL_PERIOD`.
- Second, the comparison is done assuming that `self.time` and `validity_time_end` fit in `TIMESTAMP_BITS` (which is 48 bits), but `validity_time_end` is computed as the sum of `block_timestamp` (which is 48 bits) and `MAX_ORDER_CANCEL_ALL_PERIOD` (which is equal to 240000 currently). This means that if `block_timestamp` is close to the maximum value of a 48-bit integer, the addition could overflow the `TIMESTAMP_BITS` limit, leading to the comparison being incorrect.

We also note that the comparison does not exactly assume that both operands fit within the `num_bits` limit, as specified in the documentation:

```
/// If num_bits > 48, it will use BigUintTarget for comparison. Otherwise, it
assumes that
/// inputs are in the range of 0..2^cmp_bit_size_bucket(num_bits). See
[`cmp_bit_size_bucket`]
fn is_lte(&mut self, a: Target, b: Target, num_bits: usize) → BoolTarget {
```

For 48 bits, however, `cmp_bit_size_bucket(48)` returns 48, so the concrete assumption is that both operands fit in 48 bits.

Impact. The lack of enforcement of the lower bound could allow an attacker to schedule a cancel-all operation that can be executed immediately, rather than after the intended delay. Additionally, if the `block_timestamp` is close to the maximum value of a 48-bit integer, the overflow in the addition could lead to incorrect behavior in the comparison. We note that this event is unlikely to happen in practice, as it would require the block timestamp to be almost at its maximum value.

Recommendation. We recommend implementing the lower bound check, and to increase the bit limit for the comparison to the correct number of bits, to account for the potential overflow in the addition.

Client response. Acknowledged and fixed.

#06 - Inconsistent usage of constants, commented-out code, and lingering TODOs

Severity: Informational **Location:** circuit/src/

We identified a few instances of minor code quality issues that do not pose immediate security risks but reduce long-term maintainability and clarity.

Inconsistent Usage of Constants

We noticed that constants are not always used consistently, i.e., some parts of the code use magic numbers where defined constants already exist. This makes the code harder to maintain and increases the risk of silent inconsistencies if values change in the future.

For example, the following line in `tx_constraints.rs` should use the constant `CLIENT_ORDER_INDEX_BITS` instead of 48.

```
builder.register_range_check(self.account_order_before.index_1, 48);
```

Similarly, the following line in `12_create_grouped_orders.rs` should enforce the client order to be `NIL_CLIENT_ORDER_INDEX` instead of 0.

```
builder.conditional_assert_zero(flag, self.orders[i].client_order_index);
```

Commented-Out Code

For the sake of readability, the codebase should not contain files with legacy or unused code commented out. Instead, code parts like the following in `tx_constraints.rs` should simply be removed.

```
// (  
//   [builder.zero_u8(); MAX_PRIORITY_OPERATIONS_PUB_DATA_BYTES_PER_TX],  
//   builder._false(),  
//   [builder.zero_u8(); ON_CHAIN_OPERATIONS_PUB_DATA_BYTES_SIZE],  
//   builder._false(),  
//   RegisterStackTarget::empty(builder),  
//   all_market_details_before.clone(),  
//   builder.zero_hash_out(),  
//   builder.zero_hash_out(),  
//   builder.zero_hash_out(),  
// )
```

Lingering TODOs

Currently, the codebase still contains a significant number (64) of inline “TODO” notes. While useful as reminders during development, these notes create uncertainty once the code is shipped. Given the number of these notes, it can be unclear whether they represent known limitations,

unfinished work, or low-priority improvements. For this reason, we recommend removing these notes from the code and instead converting them into items in a dedicated issue-tracking tool.

Client response. Acknowledged and partially fixed. We addressed the inconsistent usage of the above-mentioned constants and plan to make a general clean-up of the entire codebase soon. During this clean-up, we will take care of removing any commented-out code and lingering TODOs.

#07 - Executing a pop operation on an empty stack could lead to an invalid state

Severity: Informational **Location:** types/Register/register_stack.rs

Description. The `pop_front()` operation on the register stack is used to remove the top element from the stack, and return it. However, if the stack is empty, this operation will return an empty register, and the `count` variable will underflow and will be set to `-1`.

```
pub fn pop_front(&mut self, builder: &mut Builder, is_enabled: BoolTarget) {
    for i in 0..REGISTER_STACK_SIZE - 1 {
        self[i] = select_register_target(builder, is_enabled, &self[i + 1],
&self[i]);
    }
    let empty = BaseRegisterInfoTarget::empty(builder);
    self[REGISTER_STACK_SIZE - 1] =
        select_register_target(builder, is_enabled, &empty,
&self[REGISTER_STACK_SIZE - 1]);

    self.count = builder.sub(self.count, is_enabled.target);
}
```

Impact. Currently, we didn't find any immediate issues caused by this behavior, because in all instances a `pop_front()` operation is used, the stack is not empty. Additionally, assuming a correct execution of the transactions, the stack always contains an `EXECUTE_TRANSACTION` instruction at the top, which is used as a placeholder to signal an "empty" stack.

Recommendation. We recommend adding a sanity check to ensure that the stack is not empty before performing the `pop_front()` operation.

Client response. Acknowledged and fixed.

#08 - New API keys are not checked to be initialized with a zero-nonce

Severity: Informational **Location:** transactions/l2_change_pubkey.rs

Description. The L2 Change Public Key transaction allows setting new API keys for a user account, by passing an unused API key index. The transaction will store the corresponding leaf in the account's API key tree. An API key leaf contains the public key and a nonce, which is used in signature verification to prevent replay attacks.

```
pub struct ApiKeyTarget {
    pub api_key_index: Target,

    pub public_key: QuinticExtensionTarget,
    pub nonce: Target, // 48 bits
}
```

Hashing the API key leaf includes both the public key and the nonce. However, if the public key is zero, the hash returns zero, regardless of the nonce value.

```
pub fn is_empty(&self, builder: &mut Builder) → BoolTarget {
    let zero_quintic = builder.zero_quintic_ext();

    let assertions = [builder.is_equal_quintic_ext(self.public_key,
zero_quintic)];

    builder.multi_and(&assertions)
}

pub fn hash(&self, builder: &mut Builder) → HashOutTarget {
    let non_empty_hash = builder.hash_n_to_hash_no_pad::(vec![
        self.public_key.0[0],
        self.public_key.0[1],
        self.public_key.0[2],
        self.public_key.0[3],
        self.public_key.0[4],
        self.nonce,
    ]);

    let empty_hash = builder.zero_hash_out();
    let is_empty = self.is_empty(builder);

    builder.select_hash(is_empty, &empty_hash, &non_empty_hash)
}
```

This means that, when reading an empty API key (with a zero public key) from the tree, the nonce value is not checked at all, and the prover can set an arbitrary value. During a change public key

transaction, only the public key in the `api_key` object is set, which means that the arbitrary nonce value witnessed by the prover is written in the state tree.

```
impl Apply for L2ChangePubKeyTxTarget {
    fn apply(&mut self, builder: &mut Builder, tx_state: &mut TxState) →
    BoolTarget {
        tx_state.api_key.public_key =
            builder.select_quintic_ext(self.success, self.pub_key,
tx_state.api_key.public_key);

        self.success
    }
}
```

Impact. We did not find any immediate issues caused by this behavior, as the concrete nonce value is not particularly relevant for preventing replay attacks. The property that is important is that whenever a transaction is executed, the corresponding nonce is incremented.

We make, however, one remark that is somewhat independent of this specific finding. If the user is allowed to set an arbitrary public key, they could set a public key that was already used in the past. If the nonce value is reset, this could allow replaying old transactions that used the same public key. Additionally, if the user sets a duplicate public key in two different API key slots with different nonces, this could also lead to replay attacks.

Recommendation. We recommend checking that the nonce is zero when setting a new API key. As for the final remark about duplicate public keys, we recommend documenting this behavior, and preventing users from reusing the same public key in multiple API key slots and at different times.

Client response. Acknowledged and fixed.

#09 - The head of the register stack is addressed using the wrong constant

Severity: Informational **Location:** tx_constraints.rs

Description. The head of the register stack is addressed using the wrong constant, which is `OWNER_ACCOUNT_ID` instead of directly taking the first position using `[0]`, as done in other places in the code. The constant `OWNER_ACCOUNT_ID` is supposed to index the owner account inside the `accounts` vector, but it is used to index the register stack instead.

```
instruction_type: register_stack_before[OWNER_ACCOUNT_ID].instruction_type,
```

The code currently works because the `OWNER_ACCOUNT_ID` is a constant and is currently set to `0`, but it is not guaranteed to be so in the future.

Impact. This could lead to the register stack being addressed incorrectly, potentially causing issues in transaction processing and execution.

Recommendation. We recommend changing the code to directly access the first position of the register stack using `[0]`, as done in other places in the code.

Client response. Acknowledged and fixed.